use the `unstable_noStore` function from the `next/cache` module. This function allows you to opt out of caching at the component level, ensuring that your component always fetches fresh data. Here's how to implement it:

```
import { unstable_noStore as noStore } from 'next/cache';

export default async function Component() {
  noStore();
  const result = await streamText({...});
  ...
}
```

Of course, each deployment provider might have its own nuances and workarounds regarding potential problems with streaming or other considerations. In such cases, I recommend following their open issues and documentation for further insights.

8.1.2 Debugging client–server problems

Understanding the boundary between server and client components is crucial when developing web applications. In Next.js, components can be designated as either server or client components. Server components are rendered on the server and can fetch data directly, while client components are rendered in the browser and can handle user interactions.

It is essential to be able to detect any problems that happen between those boundaries. Fortunately, there are many industry-standard development tools that can help us identify and resolve problems:

- *React DevTools extension*—This powerful browser extension allows developers to inspect the React component tree, view props and state, and analyze component performance.
- *Live debuggers*—The Node.js Inspector provides a way to debug your Node.js applications using Chrome DevTools. This allows you to set breakpoints, step through code, and inspect variables in real time.
- *Console logs*—Console logging remains a fundamental debugging technique. It allows developers to output values and messages to the console, helping track the flow of execution and identify errors.

These tools should be a staple in every developer's toolkit and are essential first steps when building web apps. But not all tools are 100% bulletproof, especially when it comes to the differences in deployment providers.

Certain providers, like Vercel, do offer different Node.js runtimes as listed at their official documentation site (https://vercel.com/docs/functions/runtimes). It's no surprise that while you may be able to run your Next.js application locally without any problems, you may encounter challenges when deploying it on Vercel. For example, in its support on streaming, Vercel's documentation says: